ADAPTIVE FILE CACHE MANAGEMENT

Using Access Pattern Clustering for Workload-Aware Cache Replacement

Kevin Gilbert[1]

[1]Department of Computer Science and Engineering, Vellore Institute of Technology, Vellore, India

Abstract—

Conventional cache replacement policies use different assumptions about locality: least recently used (LRU) favors recency, least frequently used (LFU) favors accumulated frequency, most recently used (MRU) can suit scan-like access, and Adaptive Replacement Cache (ARC) balances recency and frequency. A single policy may be poorly matched to a trace whose behavior changes between sequential, looping, random, and mixed phases. This paper presents an adaptive file-cache management prototype that extracts six window-level access features, standardizes the vectors, applies a from-scratch K-Means implementation, maps heuristic workload labels to existing policies, and switches policies at observation-window boundaries. The simulator compares LRU, LFU, MRU, ARC, and the adaptive controller at 5%, 10%, and 20% cache capacities. The supplied implementation was executed with its default configuration and seed 42. Results are workload-dependent: on the mixed trace, adaptive hit ratios were 16.97%, 30.20%, and 45.40% across the three capacities, while the adaptive controller did not consistently outperform fixed baselines on looping or random traces. At 10% capacity, policy switches were 40 for looping, 3 for random, and 5 for mixed in the recorded run; frequent switching coincided with additional simulation overhead. The findings support a prototype-level conclusion: explicit workload classification can help under some access patterns, but its benefit depends on feature quality, policy mapping, and switching cost. Synthetic traces, one seed, heuristic cluster naming, and simplified costs limit generalization to production systems.

Keywords— cache replacement, adaptive caching, workload classification, K-Means, locality, LRU, LFU, MRU, ARC, operating systems.

I. INTRODUCTION

Caches reduce the cost of repeated access by keeping selected blocks in a faster storage layer. When a requested block is absent and the cache is full, a replacement policy chooses a resident block to evict. The choice matters because a hit avoids the modeled miss penalty, while a miss incurs a substantially larger cost. In the supplied simulator, a hit costs one unit and a miss costs ten units; therefore, even modest changes in hit ratio can change average access cost.

LRU, LFU, MRU, and ARC embody different assumptions about access locality. LRU uses recency, LFU uses access frequency, MRU evicts the newest resident entry, and ARC adapts its balance between recency and frequency. Workloads are not stationary in general. A trace may transition from scans to repeated loops or from concentrated reuse to broad random access. A policy selected for one phase can perform poorly after the phase changes.

This project investigates whether window-level characterization can support a transparent policy-selection layer above conventional replacement algorithms. The implementation is a simulator prototype, not a kernel patch or production storage service. It uses synthetic request traces, six engineered features, K-Means clustering, heuristic cluster names, and a configurable policy map.

- C1 — Integrated prototype: request generation, feature extraction, clustering, policy selection, cache simulation, and metrics are connected in a reproducible pipeline.
- C2 — Multiple policy baselines: the adaptive controller is compared with LRU, LFU, MRU, and ARC on identical traces and capacities.
- C3 — Workload-level analysis: sequential, looping, random, and mixed traces expose cases where a mapping succeeds and cases where it does not.
- C4 — Overhead visibility: policy switches and adaptation time are reported alongside hit and miss behavior.
- C5 — Explicit limitations: synthetic inputs, one default seed, heuristic labels, and simplified access costs are treated as threats to external validity.

II. RELATED WORK AND RESEARCH GAP

A. Recency- and Frequency-Based Replacement

LRU approximates temporal locality by evicting the least recently referenced item. LFU favors items with larger accumulated counts, but its behavior can be sensitive to stale popularity unless aging is included. MRU is counterintuitive for cyclic scans and can be useful when recently accessed blocks are less likely to be reused immediately. These policies are inexpensive and interpretable, but their assumptions differ. The project includes them as fixed baselines rather than claiming to improve their internal eviction rules [1], [2].

B. Adaptive Replacement and Online Policy Selection

ARC combines recency- and frequency-oriented resident lists with nonresident history to adapt its target balance as workload behavior changes [3]. Online methods such as LeCaR use feedback to select between candidate policies [4]. The present project differs in control granularity: it classifies an observation window and selects a complete replacement policy for the next window. This is easier to inspect, but can be slower and more brittle than adaptation performed inside an eviction algorithm.

C. Workload Characterization and Locality

Access traces can be summarized through working-set size, reuse distance, sequentiality, and concentration of requests among popular blocks. Such statistics are not interchangeable. A large working set may still contain a highly reused hot subset; a high sequentiality score describes adjacent identifiers rather than application semantics. The feature vector therefore represents proxies for locality, not a complete description of the workload.

D. Clustering as a Workload Labeler

K-Means partitions standardized vectors by squared Euclidean distance and assigns each observation to its nearest centroid [5]. It does not inherently know the labels ‘sequential’ or ‘looping’; those labels are assigned after clustering using hand-written rules over feature profiles. Cluster IDs can change between runs, and a trace may not contain every semantic class. A requested cluster count therefore should not be interpreted as proof that each named class was discovered.

E. Learning-Based and Predictive Cache Research

Learning-based cache replacement research has explored policies that predict reuse or combine existing algorithms. These approaches establish that data-driven cache management is a broader field, not a novel concept introduced here. The current contribution is narrower: a lightweight, interpretable classification layer that maps window-level features to familiar policy implementations and measures the consequences in a simulator.

F. Evaluation Practices and Trace Validity

A replacement policy can appear effective when the generator creates a narrow or highly periodic workload. Results on synthetic traces are useful for debugging and controlled comparisons but do not establish production benefit. This report records the default-seed experiment and distinguishes observed measurements from hypotheses. Repeated seeds, public traces, confidence intervals, and stronger baselines remain necessary for a general claim.

G. Research Gap and Problem Formulation

The practical question is whether explicit window-level classification can select a policy well enough to offset clustering and switching overhead. Let a trace be x = (x■, x■, …, x■), divided into windows of length W. Each window is represented by a feature vector, assigned to a cluster, mapped to a policy, and then replayed through the cache simulator. The experiment compares the adaptive result with fixed policies on the same requests.

TABLE I — COMPARISON OF CACHE-POLICY FAMILIES

Policy	Primary signal	Strength	Known risk	Role here
LRU	Recent access order	Simple temporal locality	Scans can evict useful blocks	Fixed baseline
LFU	Access counts	Repeated popularity	Stale counts / phase changes	Fixed baseline
MRU	Most recent item	Can suit scan-like streams	Poor under many reuse patterns	Fixed baseline
ARC	Recency + frequency history	Self-adjusting balance	More state/complexity	Fixed baseline
Adaptive	Cluster label → policy	Explicit phase response	Misclassification/switch cost	Proposed controller

III. SYSTEM ARCHITECTURE

The architecture separates observation from policy execution. A trace generator creates a stream of block identifiers. The stream is divided into windows, and each window is summarized by six features. Standardized feature rows are clustered with K-Means. Heuristic rules assign semantic labels to cluster profiles, and a configurable mapping selects LRU, LFU, MRU, or ARC. The selected policy serves requests for the next window; evaluation records hit ratio, miss ratio, access cost, switch count, and timing.

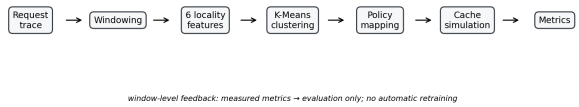


Fig. 1. Architecture of the adaptive file-cache management prototype: trace generation, window features, clustering, policy mapping, simulation, and metrics.

At a policy boundary, the controller extracts resident entries from the current policy object and initializes the new policy with those entries. This preserves cache membership to the extent supported by the implementation, but policy-specific ordering and metadata are rebuilt. Consequently, a switch can change future eviction order even when the resident set is initially unchanged.

TABLE II — CONFIGURATION AND TIMING QUANTITIES

Parameter	Default	Interpretation
Requests	50,000	Synthetic trace length
Block universe	5,000	Identifier range for generation
Window size	1,000	Requests summarized per observation
Clusters	4	Requested K-Means centroids
K-Means restarts	8	Initializations evaluated
Max iterations	100	Per-initialization limit
Hit / miss cost	1 / 10	Fixed modeled access costs
Cache fractions	5%, 10%, 20%	Relative to trace working set

A. Data-Flow Boundaries

Feature extraction and cluster assignment operate on windows rather than individual cache requests. Policy selection therefore reacts at window boundaries. The design trades immediate responsiveness for lower decision frequency and a more stable feature summary. A window that contains a phase transition can mix two behaviors, while a short phase may end before the controller changes policy.

B. State and Feedback

Feedback in this prototype means metrics are collected and exported for analysis. There is no automated model retraining and no online optimization of the policy mapping. The current architecture should not be described as a production feedback controller; it is a repeatable simulation loop that exposes policy transitions.

IV. METHODOLOGY

A. Trace Generation and Windowing

Four workload families are generated: sequential access, looping reuse, random access, and a mixed trace combining behaviors. The experiment uses deterministic seed 42. A window contains 1,000 requests by default. The generator provides controlled workloads for debugging and comparison, but its traces are not substitutes for application or file-system traces.

B. Telemetry and Feature Vector

Each observation window is converted into six values. Unique ratio measures distinct blocks per request. Sequentiality estimates the fraction of adjacent requests whose block identifiers differ by at most one. Reuse distance summarizes the positional gap between repeated accesses. Frequency concentration measures the request share contributed by the hottest 20% of distinct blocks. Inter-arrival time is implemented using the same mean reuse-distance proxy, making the two features correlated. Working-set size is the number of distinct blocks in the window.

unique_ratio	distinct blocks / requests
sequentiality	adjacent block IDs within distance 1
reuse_distance	mean position gap between repeats
frequency_concentration	request share of hottest 20% blocks
inter_arrival_time	implemented as mean reuse-distance proxy
working_set_size	number of distinct blocks in window

Fig. 2. Six window-level features and their implemented interpretations.

TABLE III — FEATURE DEFINITIONS AND INTERPRETATION

Feature	Operational definition	High value may indicate
unique_ratio	distinct / requests	Broad exploration
sequentiality	adjacent IDs within ±1	Spatial adjacency
reuse_distance	mean positional gap	Longer repeat gaps
frequency_concentration	share from hottest 20%	Skew / hot subset
inter_arrival_time	same proxy as reuse distance	Correlated repeat timing
working_set_size	distinct blocks	Larger window footprint

C. Standardization and Clustering

Feature columns are standardized using their observed mean and population standard deviation. K-Means repeatedly assigns each vector to its nearest centroid and updates centroids until convergence or the iteration limit. Multiple initializations are evaluated, and the solution with the lowest inertia is retained. Standardization prevents features with larger numeric scales from dominating the Euclidean distance solely because of units.

For feature vector z_i and centroid μ_k, the assignment is:

$$k^*(i) = \arg\min_k \|z_i - \mu_k\|^2 \quad (1)$$

The objective minimized by K-Means is:

$$J = \sum_i \|z_i - \mu_{k^*(i)}\|^2 \quad (2)$$

D. Semantic Labeling

Cluster IDs are anonymous. The implementation labels one cluster sequential when sequentiality is high, then identifies looping and random profiles using frequency concentration, reuse-distance, unique-ratio, and sequentiality heuristics. Remaining clusters are labeled mixed. These rules are simple and inspectable but are not a learned classifier; the resulting labels depend on the feature distributions in the current experiment.

E. Leakage and Reproducibility Controls

The project is not a supervised forecasting pipeline, so there is no target-label leakage analysis analogous to a prediction study. Reproducibility instead depends on fixed seeds, deterministic trace generation, consistent cache capacities, and identical request streams for each baseline. The report's numerical results are from the supplied implementation under its default seed, not invented measurements. One seed does not quantify variance across traces.

V. ADAPTIVE POLICY DECISION AND EXPLANATION

A. Cluster-to-Policy Mapping

The default mapping is sequential → MRU, looping → LRU, random → LFU, and mixed → ARC. It encodes a hypothesis about which policy may suit each detected access pattern. The mapping is configurable and should be tested against held-out traces; it is not an established optimal mapping.

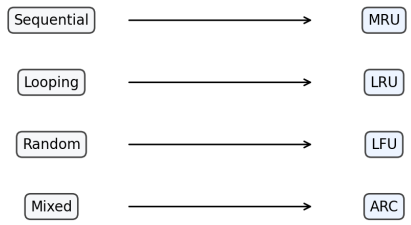


Fig. 3. Default heuristic mapping from workload label to replacement policy.

TABLE IV — DEFAULT POLICY MAPPING

Label	Selected policy	Intended rationale	Risk
Sequential	MRU	Recent item may not be reused	Sequentiality proxy is crude
Looping	LRU	Recency tracks loop reuse	Working set may exceed cache
Random	LFU	Popular items may recur	Random trace may lack skew
Mixed	ARC	Balances recency/frequency	Switches may be noisy

B. Policy Switching

At the beginning of a window, the controller reads its cluster label and resolves the selected policy. If the target differs from the active policy, the resident cache entries are transferred to a new policy instance and the switch counter increments. If the target is unchanged, no rebuild occurs. The implementation does not include hysteresis, minimum dwell time, or a cost-aware switch threshold.

C. Hit, Miss, and Cost Metrics

For N requests, let H denote hits and M misses, with H + M = N. Hit ratio and miss ratio are:

$$HR = H/N;\ MR = M/N = 1 - HR \quad (3)$$

With hit cost $C_h = 1$ and miss cost $C_m = 10$, average access cost is:

$$C_h = (H \cdot C_h + M \cdot C_m)/N = 1 \cdot HR + 10 \cdot MR \quad (4)$$

D. Overhead and Graceful Behavior

Clustering time and adaptive simulation time are measured separately. The adaptation timer includes the simulation path used by the implementation and varies with cache size, workload, and the number of policy changes. Timing values are environment-dependent and should not be treated as production latency. Frequent switching can incur extra state-transfer work without increasing hit ratio.

E. Feature-Based Explanation

The feature vector provides a lightweight description of why a window received a particular cluster label: sequentiality, uniqueness, reuse distance, concentration, and working-set size can be inspected directly. This is not a post-hoc causal explanation of an eviction decision. A stronger explanation layer would report the distance to centroids, the features responsible for the winning label, and uncertainty when two cluster profiles are similar.

TABLE V — POLICY-SWITCH CONTROLLER STATES

Condition	Action	Recorded effect
Target = active policy	Continue current policy	No switch increment
Target differs	Rebuild policy around residents	Switch count +1
Unknown label / fallback	Use configured fallback	Potential label-mapping error
Window changes mid-phase	Wait for next boundary	Detection delay up to one window

VI. EXPERIMENTAL SETUP

A. Metrics and Protocol

The evaluator runs LRU, LFU, MRU, ARC, and ADAPTIVE on each generated trace at three cache fractions. Reported metrics are hit ratio, miss ratio, average access cost, policy-switch count, clustering time, and adaptation time. Every fixed baseline sees the same trace and capacity as the adaptive controller. Cache capacity is based on the unique-block working set generated for the trace.

B. Workload Families

TABLE VI — WORKLOAD FAMILIES

Workload	Designed behavior	Question tested
Sequential	Adjacent/increasing block access	Can scan-like locality be recognized?
Looping	Repeated working-set cycle	Can reuse be retained?
Random	Broad sampled identifiers	Does frequency help?
Mixed	Phases combine multiple patterns	Can classification help during transitions?

C. Cache Capacities and Cost Model

The evaluated capacities are 5%, 10%, and 20% of the trace's unique-block working set. Hit cost is fixed at one and miss cost at ten. This deliberately simple model allows transparent comparisons but omits latency distributions, writes, dirty-block costs, device queues, CPU contention, and concurrent workloads.

TABLE VII — EXPERIMENT MATRIX

ID	Workload	Policies	Capacity settings	Repetitions
E1	Sequential	5	5/10/20%	1 seed
E2	Looping	5	5/10/20%	1 seed
E3	Random	5	5/10/20%	1 seed
E4	Mixed	5	5/10/20%	1 seed

D. Implementation and Execution

The implementation is separated into configuration, trace generation, feature extraction, clustering, cache-policy implementations, adaptive switching, simulation, evaluation, plotting, and experiment orchestration. CSV outputs and figures support inspection. The results in this paper were generated by executing the supplied code with default configuration and seed 42. No real file-system trace or hardware benchmark was used.

TABLE VIII — REPORTED METRICS

Metric	Definition	Interpretation
Hit ratio	H / N	Higher is better
Miss ratio	M / N	Lower is better
Average cost	(H + 10M) / N	Lower is better
Switch count	Policy changes at windows	Lower is not always better
Clustering time	Feature clustering runtime	Environment-dependent
Adaptation time	Adaptive simulation runtime	Includes policy switching path

VII. RESULTS AND DISCUSSION

A. Default Run and Workload Labels

The sequential trace yielded one observed semantic cluster in the recorded run, whereas looping, random, and mixed traces yielded three or four labels. This illustrates that requesting four centroids does not guarantee four meaningful classes. The labels are heuristic summaries of generated windows, not ground-truth annotations.

TABLE IX — OBSERVED CLUSTER LABELS

Workload	Observed labels	Interpretation
Sequential	sequential	One dominant access profile
Looping	sequential, looping, random, mixed	Several window profiles detected
Random	sequential, looping, random	Not all requested labels present
Mixed	sequential, looping, random, mixed	All four heuristic labels appeared

B. Mixed Workload: Hit Ratio

The mixed trace is the strongest observed case for the adaptive mapping. Adaptive hit ratios were 16.97%, 30.20%, and 45.40% at 5%, 10%, and 20% capacity. In the executed run, the best fixed hit ratios at those capacities were approximately 10.71%, 24.92%, and 37.61%, respectively. This is a result for one synthetic trace and one seed, not evidence of universal superiority.

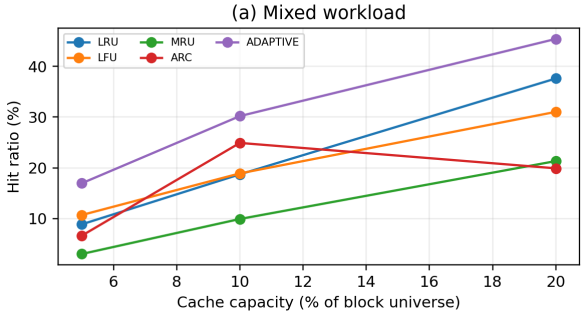


Fig. 4. Hit ratio versus cache capacity for each fixed policy and the adaptive controller on the mixed trace.

TABLE X — MIXED-TRACE HIT RATIO (%)

Capacity	LRU	LFU	MRU	ARC	Adaptive
5%	8.88	10.71	3.03	6.65	16.97
10%	18.72	18.91	9.93	24.92	30.20
20%	37.60	31.03	21.34	19.91	45.40

C. Mixed Workload: Access Cost

Because the cost model assigns a tenfold cost to misses, hit-ratio improvements reduce average cost. Adaptive average costs in the mixed trace were approximately 8.48, 7.28, and 5.91 units at the three capacities. The relationship is arithmetic under the chosen model; it does not imply a measured latency reduction on real storage.

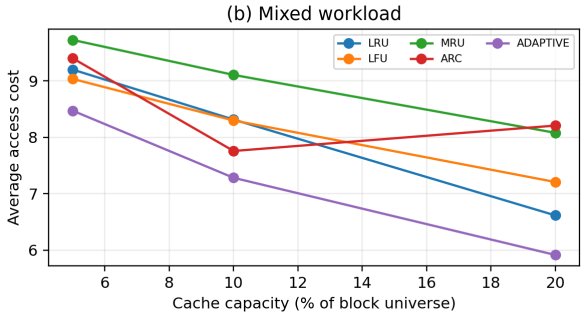


Fig. 5. Average modeled access cost by cache capacity on the mixed trace.

TABLE XI — MIXED-TRACE AVERAGE ACCESS COST

Capacity	LRU	LFU	MRU	ARC	Adaptive
5%	9.201	9.036	9.727	9.401	8.472
10%	8.315	8.298	9.107	7.758	7.282
20%	6.616	7.207	8.080	8.208	5.914

D. Cross-Workload Comparison

The adaptive controller does not dominate every baseline. At 10% capacity, the looping trace's adaptive hit ratio was about 5.58%, below LFU (11.56%) and ARC (11.86%). On the random trace, adaptive reached about 5.52%, approximately matching but not exceeding the LFU/ARC value of 5.64%. On the sequential trace, adaptive matched MRU under the configured mapping. These outcomes show that a policy mapping can be sensible for one trace and mismatched for another.

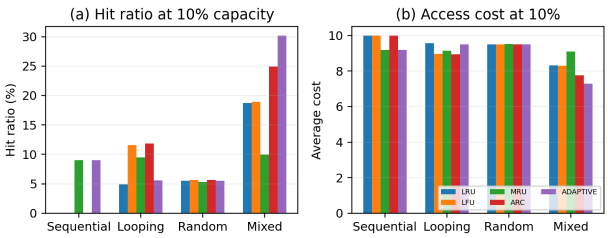


Fig. 6. Two-panel comparison of hit ratio and average cost at 10% capacity across the four workload families.

TABLE XII — HIT RATIO AT 10% CAPACITY (%)

Workload	LRU	LFU	MRU	ARC	Adaptive
Sequential	0.00	0.00	9.00	0.00	9.00
Looping	4.91	11.56	9.52	11.86	5.58
Random	5.48	5.64	5.28	5.64	5.52
Mixed	18.72	18.91	9.93	24.92	30.20

E. Capacity Sensitivity

Increasing cache capacity improved hit ratio for the tested workloads, as expected when a larger set of blocks can remain resident. The slope and absolute performance varied by trace. Capacity alone did not eliminate policy mismatch: adaptive performance on looping remained below stronger fixed baselines in the recorded run.

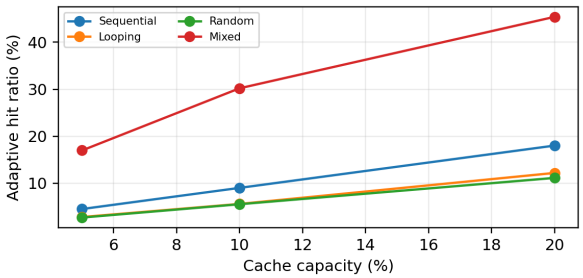


Fig. 7. Adaptive hit ratio as cache capacity increases across sequential, looping, random, and mixed traces.

TABLE XIII — ADAPTIVE HIT RATIO BY WORKLOAD (%)

Workload	5%	10%	20%
Sequential	4.50	9.00	18.00
Looping	2.80	5.58	12.18
Random	2.68	5.52	11.12
Mixed	16.97	30.20	45.40

F. Switching Frequency and Runtime

At 10% capacity, the adaptive controller recorded one switch for sequential, 40 for looping, three for random, and five for mixed in the current run. The looping trace's 40 switches are a warning that cluster labels can change frequently across windows. More switches increase adaptation time and may be counterproductive when the newly selected policy does not improve locality.

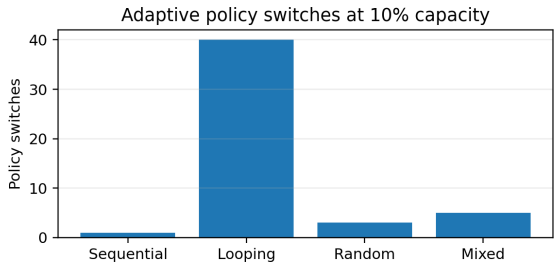


Fig. 8. Policy-switch counts at 10% cache capacity in the default run.

TABLE XIV — ADAPTIVE SWITCH COUNTS AND RUNTIME AT 10%

Workload	Switches	Adaptation time (ms)	Observed implication
Sequential	1	17.7	Single dominant class
Looping	40	342.5	Frequent policy changes
Random	3	18.2	Few detected transitions
Mixed	5	757.9	Multiple phase changes

G. Policy-Selection Behavior

The adaptive controller follows the configured mapping rather than optimizing policy choice online. Its behavior is therefore constrained by two decisions: whether the feature pipeline labels the window appropriately, and whether the mapping associates that label with a policy suited to the trace. A wrong label and a wrong mapping have similar visible effects but require different remedies.

H. Fixed Versus Adaptive Evaluation

The fair comparison is per workload and capacity, not by averaging away failures. Adaptive is best on the observed mixed trace at all three capacities, matches MRU on the sequential trace, and underperforms stronger fixed policies on looping. On random access, results are close and the adaptive controller does not show a clear benefit. No statistical significance claim is made because only one seed is reported.

TABLE XV — WINNER BY WORKLOAD AND CAPACITY (OBSERVED RUN)

Workload	5% best hit ratio	10% best hit ratio	20% best hit ratio
Sequential	MRU / Adaptive	MRU / Adaptive	MRU / Adaptive
Looping	ARC	ARC	MRU
Random	LFU / ARC	LFU / ARC	LFU / ARC
Mixed	Adaptive	Adaptive	Adaptive

I. Explanation Reliability

The current feature vector is interpretable as a set of summary statistics, but the system does not provide calibrated confidence or a formal explanation for a cluster assignment. In particular, inter-arrival time and reuse distance are redundant in the implementation. A future explanation should expose normalized feature values, centroid distances, and whether the selected label is stable under small perturbations.

TABLE XVI — EXPLANATION AND DIAGNOSTIC CHECKLIST

Diagnostic	Current status	Recommended validation
Feature values	Available per window	Inspect distributions by workload
Cluster label	Heuristic	Compare with known generator phase
Centroid distance	Not emphasized in output	Report nearest/second-nearest gap
Feature redundancy	Known: reuse metrics overlap	Remove or redefine correlated feature
Mapping rationale	Fixed table	Ablate each mapping choice

J. Inference and Switching Cost

K-Means clustering is performed over the observed feature windows, and adaptive simulation runtime includes policy changes. Reported timing depends on the runtime environment and trace size. A deployment claim would require profiling feature extraction and switching separately, comparing total cost with the benefit from misses avoided, and evaluating on real traces. No such production measurements are available in the supplied materials.

VIII. WHAT IS NEW AND WHAT IS NOT SHOWN

The project is an integrated educational prototype combining six-feature window summaries, a from-scratch K-Means implementation, heuristic semantic labeling, a configurable mapping to four conventional policies, and a simulator that reports hit/miss behavior and switching overhead. The integration and transparent evaluation are the project's practical contributions.

The project does not invent LRU, LFU, MRU, ARC, K-Means, or adaptive caching. It does not establish that the proposed mapping is optimal, that the model generalizes to production traces, or that adaptive selection reduces real storage latency. It does not implement kernel-level cache management, concurrency control, dirty-page handling, or a real-time operating-system controller.

TABLE XVII — IMPLEMENTED SCOPE, TESTED SCOPE, AND FUTURE WORK

Area	Implemented / tested	Not demonstrated
Trace input	Synthetic sequential/looping/random/mixed	Production trace generalization
Features	Six window-level features	Independent predictive value of each feature
Clustering	K-Means + heuristic labels	Ground-truth classification accuracy
Policies	LRU, LFU, MRU, ARC	Kernel integration or concurrency
Adaptation	Window-boundary switching	Optimal online policy selection
Evaluation	Hit/miss/cost/switch/timing	Hardware latency or energy savings

IX. LIMITATIONS AND THREATS TO VALIDITY

- Trace validity: all experiments use synthetic workload generators; application and production file-system traces were not tested.
- Statistical validity: results are from one default seed. No confidence intervals, multi-seed mean, or significance tests are reported.

APPENDIX A. EXTENDED IMPLEMENTATION AND EVALUATION NOTES

Trace representation

The trace is an ordered stream of integer block identifiers, not a physical disk image. The simulator sees the same request order for every policy, which isolates replacement behavior from unrelated variables. A hit occurs when the identifier is resident; a miss occurs otherwise. On a full-cache miss, the active policy chooses a victim. This abstraction is useful for controlled experiments, but it does not model request merging, asynchronous I/O, dirty pages, block sizes, or device scheduling. Results should therefore be interpreted as policy-simulation outcomes.

Window size

The default window contains 1,000 requests. Short windows react quickly but produce noisier feature estimates, while long windows smooth noise at the cost of mixing different workload phases. A phase that lasts less than one window may not be represented independently. Window size should be swept over several values, with trace and cache capacity held fixed, to quantify this responsiveness-stability trade-off. The present value is a configuration choice rather than a tuned optimum.

Unique ratio

Unique ratio is the number of distinct blocks divided by request count within a window. A high value suggests that few requests repeat locally; a low value indicates repeated access. It cannot distinguish a tight loop from a highly skewed hot-key workload on its own. Its interpretation depends on window length and should be combined with concentration, reuse distance, and working-set size. It is a summary of the observed window, not a prediction of future reuse.

Sequentiality proxy

The implementation counts adjacent request pairs whose identifiers differ by at most one. This is intentionally cheap to compute and works with the synthetic generator's block numbering. However, numerical adjacency is not always physical or logical locality in a real storage trace. In a production evaluation, the metric should use block addresses, offsets, or known sequential-read indicators. A robustness check should also vary the adjacency threshold to determine whether labels depend on this arbitrary boundary.

Reuse distance

Reuse distance summarizes the positional gap between repeated references. Small gaps can indicate that recency-based policies will retain useful items, whereas large gaps can indicate weak immediate reuse. An average loses distributional information: two windows can have the same mean but very different mixtures of short and long reuse intervals. Histograms, medians, and upper quantiles could improve characterization. The present implementation keeps the feature vector small for interpretability and low computational cost.

Frequency concentration

Frequency concentration estimates the share of requests attributable to the hottest 20% of distinct blocks. It aims to identify skewed workloads with a relatively small popular subset. This measure is related to, but not equivalent to, entropy, Gini concentration, or a Zipf parameter. The hottest fraction is fixed, so the statistic can behave differently when working-set sizes vary widely. Future experiments should compare alternative concentration metrics and test whether each adds information beyond unique ratio.

Correlated features

The implemented inter-arrival-time feature reuses the mean reuse-distance calculation. These two columns therefore carry duplicate or strongly overlapping information. Since K-Means uses Euclidean distance, duplicated information can receive extra weight and influence centroid placement. This is a concrete design limitation. A useful ablation would remove one of the two features and compare cluster assignments, switch counts, hit ratios, and runtime under identical seeds. If results remain unchanged, the simpler vector is preferable.

Standardization

Feature standardization subtracts each feature's mean and divides by its population standard deviation. It reduces scale dominance when, for example, working-set size is measured in hundreds while unique ratio lies between zero and one. Zero-variance columns require special handling to avoid division by zero. If the model is deployed online, the scaler parameters must remain stable or be updated under an explicit protocol; otherwise, the same access window may map to a different region of feature space.

K-Means objective

K-Means assigns each feature vector to its nearest centroid under squared Euclidean distance and updates each centroid to the mean of assigned points. It seeks a compact partition, not necessarily semantically meaningful classes. Multiple restarts reduce sensitivity to initialization but cannot guarantee a global optimum. Cluster quality should be assessed using inertia, stability across seeds, and alignment with known generator phases. A low clustering objective alone does not show that the selected cache policy will perform well.

Cluster identifiers

Numeric cluster identifiers have no fixed meaning. Cluster 0 in one run can represent a different profile from cluster 0 in another run. The implementation assigns semantic names after clustering using heuristics, which helps interpret plots but creates another source of uncertainty. Stable deployment would require a consistent naming protocol based on centroid profiles, or a fixed online assignment model. Any change in clustering parameters should trigger revalidation of the label-to-policy map.

Sequential label

The sequential label is assigned from the sequentiality signal. This can be reasonable for the generated sequential workload, but adjacency between integer identifiers is only a proxy. A trace may contain locally adjacent requests mixed with frequent revisits to a hot subset. In such a case, sequentiality alone can mask a reuse pattern. Evaluation should report feature distributions by known workload phase and show examples of windows where the heuristic label differs from the generator's intended class.

Looping label

The looping label is based on reuse and frequency-related features. A looping workload can still behave poorly when its active working set exceeds cache capacity. It may also look like a mixed workload if the generator includes occasional jumps or scans. Therefore, assigning LRU to a looping label is a hypothesis, not a guarantee. A controlled test should vary loop length relative to cache capacity and measure whether the selected policy remains beneficial across those ratios.

Random label

The random label is intended to represent broad or weakly local access. If the random generator is uniform, frequency-based policies may offer little advantage because popular blocks are not much more common than others. If the generator is skewed, LFU may perform better. Thus, random access should be evaluated across both uniform and heavy-tailed distributions. The current four-family setup is useful for a first comparison but does not cover the full range of real workloads.

Mixed label

The mixed label is used for windows that do not fit the sequential, looping, or random heuristics. It is a residual category rather than a rigorously defined class. The label can collect multiple unrelated patterns, which may require different policies. A future classifier should distinguish the most common mixed subtypes or let the controller retain the current policy when evidence is ambiguous. This would avoid treating every unclassified window as a reason to switch to ARC.

LRU baseline

LRU evicts the least recently used resident entry. It is a natural baseline because recency is often correlated with future reuse and its behavior is easy to explain. A scan can, however, push useful blocks out of the cache. LRU performance depends on cache capacity relative to the active working set. The project compares the fixed LRU implementation with the adaptive controller on identical requests, so any observed difference is due to policy behavior and switching rather than workload differences.

LFU baseline

LFU evicts according to accumulated access counts and can retain frequently reused items. It may be strong when popularity is stable, but stale counts can be harmful after phase changes unless aging is implemented. Different LFU variants use different tie-breaking and decay rules, so results are specific to the supplied implementation. The policy should not be described as a single universal algorithm; the exact count and tie-breaking behavior belongs in the reproducibility record.

MRU baseline

MRU removes the most recently used resident item when replacement is required. This can be useful for particular scan or cyclic patterns where the item just accessed is unlikely to be reused soon. It can be poor when recent items are likely to recur. The sequential-to-MRU mapping in this project is an experimental choice. A real sequential workload should be tested rather than assumed to benefit from MRU based solely on the label.

ARC baseline

ARC adapts between recency and frequency using resident and ghost-history structures. It is included because it already responds to workload behavior internally and is a stronger comparator than only static policies. The project's implementation is a compact simulator implementation; it should be checked against a trusted reference implementation before making broad claims. The adaptive controller should demonstrate a benefit beyond ARC's internal adjustment to justify the added feature and clustering layer.

Policy mapping

The configured map selects MRU for sequential, LRU for looping, LFU for random, and ARC for mixed. The map is transparent and easy to modify, but it is manually specified. The observed results show that the same mapping does not win across all workload families. A fair follow-up should tune the mapping on training traces and report results on unseen test traces, avoiding selection of a mapping solely from the same traces used for evaluation.

Switch state migration

On a policy change, resident entries are transferred to a new policy instance. This avoids an intentional full-cache flush, but policy-specific metadata cannot always be transferred exactly. Reinserted items may acquire new recency order or frequency state. The transition can therefore alter future evictions independently of the selected policy's inherent behavior. A detailed state-migration test should exercise every policy pair and check both cache membership and subsequent victim choices.

Hit ratio

Hit ratio is hits divided by requests. It is a direct metric of how often the cache avoids the modeled miss penalty, but it does not capture CPU overhead or latency variance. Policies with equal hit ratio can still have different metadata costs and synchronization behavior. The simulator focuses on hit ratio because its model is intentionally simple. A production study should include throughput, latency percentiles, memory overhead, and policy-management cost.

Miss ratio

Miss ratio is misses divided by requests and equals one minus hit ratio when every request is classified as exactly one hit or miss. It is useful for sanity-checking result tables: hit and miss ratios should sum to one, subject to rounding. A reported improvement in hit ratio should correspond to the same absolute decrease in miss ratio. If those values disagree, the evaluation pipeline should be checked before interpreting the plots.

Modeled access cost

Average access cost weights each hit by one unit and each miss by ten units. It is a model-derived score, not milliseconds. The fixed tenfold penalty makes a miss reduction look substantial by construction. A sensitivity analysis should vary the miss penalty and include switching overhead in the same cost objective. Otherwise, a policy can appear beneficial on hit ratio while consuming more compute than it saves in access cost.

Capacity normalization

The 5%, 10%, and 20% capacities are relative to the unique-block working set of each trace. Since workload families can generate different numbers of unique identifiers, the actual number of cache slots can differ at the same percentage. Cross-workload comparisons should report both percentage and absolute capacity. Normalizing by unique blocks makes the experiments comparable in relative working-set terms, but not identical in memory footprint.

Sequential result

The sequential trace produced one observed semantic label, and the adaptive controller selected MRU under the mapping. Its hit ratio matched MRU. This is a useful consistency check, but it does not show that K-Means was necessary: a simple sequentiality threshold might select MRU with lower overhead. Future work should include a rule-only controller as an ablation and compare both performance and computational cost.

Looping result

The looping trace produced many policy transitions. At 10% capacity, the adaptive controller recorded 40 switches and achieved a lower hit ratio than LFU and ARC in the observed run. One interpretation is that noisy labels or the chosen mapping caused policy churn; another is that state reconstruction disrupted useful metadata. The experiment does not isolate these mechanisms. A switch timeline aligned with misses and cluster labels is needed before attributing the loss to one cause.

Random result

At 10% capacity, adaptive hit ratio on the random trace was close to LFU and ARC, but did not clearly exceed them. This is expected when the trace has little exploitable temporal structure. A richer random-workload experiment should vary skew, hot-set size, and phase duration. It should also compare whether the classifier can distinguish uniform random traffic from skewed random traffic, since the latter may benefit from frequency-aware policies.

Mixed result

The adaptive controller had the strongest relative result on the mixed trace, exceeding the best fixed baseline at the three capacities in the recorded run. This supports the possibility that window-level selection can help when a trace changes access pattern. However, one seed cannot establish generality. A multi-seed evaluation should include varied transition points and phase lengths, and report confidence intervals for the paired hit-ratio difference against the best fixed baseline.

Switch count

Switch count is a diagnostic rather than an objective by itself. A controller can switch often because real workload phases change, or because cluster labels are unstable. The looping result suggests that switch frequency deserves explicit attention. Hysteresis, minimum dwell time, or a switch penalty could reduce churn, but may delay response to genuine changes. These controls should be evaluated through a sweep rather than added without measurement.

Timing measurements

Adaptation time is affected by trace length, cache capacity, Python runtime, policy-state transfer, and the number of switches. It is not directly comparable to a kernel decision-latency claim. Timing results should be repeated on a documented machine and reported with dispersion. Feature extraction, clustering, state migration, and cache simulation should be timed separately to identify the actual overhead source.

Statistical replication

A single deterministic seed is useful for reproducing a bug or exact trace but does not measure variability. A stronger protocol would run multiple independent seeds per workload and capacity, calculate paired differences against each fixed baseline, and report mean, standard deviation, and confidence intervals. The same traces should be used across policies to maintain a fair comparison. Negative results should be retained rather than excluded.

Real-trace evaluation

Synthetic workloads are controlled but may omit realistic phase transitions, skew, concurrent clients, and I/O dependencies. A next step is to use public block traces or application-generated traces, documenting preprocessing and anonymization. Test traces should remain separate from any tuning process. The goal is not only higher average hit ratio but also stable performance under traces that differ from the synthetic generator.

Online deployment

A live controller would need to assign each new window using stable centroids and maintain a consistent mapping between clusters and policies. It would also need safeguards against rapid switching, missing telemetry, and invalid feature values. The current prototype does not implement kernel integration, concurrency control, or operational rollback. Those engineering requirements are separate from demonstrating a simulation-level hit-ratio result.

Cost-aware switching

The current mapping switches whenever the label maps to a different policy. It does not estimate whether the expected improvement is worth the cost of rebuilding policy state. A future controller could require an expected cost reduction greater than a switch penalty or run small shadow caches to estimate candidate performance. Both ideas add overhead and should be compared with the current simple rule.

Feature ablation

Ablation should test each feature's contribution to cluster stability and policy outcome. Because two features are correlated, removing one may reduce duplicated weight without harming discrimination. Results should include the number of switches, label agreement with known phases, hit ratio, and runtime. A feature that improves cluster separation but increases switching or fails to improve cache outcomes may not be useful to the controller.

Reproducibility

The default run uses 50,000 requests, 5,000 possible identifiers, seed 42, a 1,000-request window, four requested clusters, 100 K-Means iterations, eight restarts, capacities of 5%, 10%, and 20%, and costs of one for a hit and ten for a miss. The report should be accompanied by raw CSV outputs and source files. The reported numerical metrics are simulator outputs, not externally benchmarked storage measurements.

Threat interpretation

The principal threats are synthetic trace validity, one-seed variability, heuristic labels, feature redundancy, manually selected mapping, and the simplified cost model. These limitations do not invalidate the implementation; they bound the claims that can be made from it. The strongest defensible conclusion is that the prototype works end-to-end and that its performance is workload-dependent, not that it universally outperforms established cache policies.

Future work priority

The most valuable next steps are to remove the duplicated feature, validate labels against generator phases, add hysteresis, run multiple seeds, and test on public traces. These changes address the largest threats to validity before adding more complex learning methods. Only after demonstrating stable benefit should the project consider real operating-system integration or claims about production latency.

APPENDIX B. EXTENDED RESULTS INTERPRETATION AND VALIDATION PLAN

B.1 Sequential trace at 5% capacity

At 5% capacity, the sequential trace's adaptive result matches MRU because the observed windows map to the sequential label and that label selects MRU. The other fixed policies show near-zero hits in the recorded trace, illustrating how a scan-like stream can defeat recency/frequency choices under this generator. The comparison is specific to the request order and block-ID construction; it should not be generalized to all sequential workloads.

B.2 Sequential trace at 10% capacity

Increasing capacity from 5% to 10% improves MRU and adaptive hit ratio because more recently scanned identifiers remain available. The adaptive controller does not add a performance benefit over MRU in this case. This is a useful negative control: the classification layer can identify a consistent pattern, but when one fixed policy already matches that pattern, adaptation adds complexity without an observed hit-ratio gain.

B.3 Sequential trace at 20% capacity

At 20% capacity, MRU and adaptive again match. The result indicates that capacity growth improves the absolute hit ratio for this trace, but the relative ranking remains unchanged. A future experiment should vary the scan stride and include periodic revisits to older blocks. Such traces would clarify whether the current sequentiality proxy distinguishes a pure scan from a scan with a reusable hot subset.

B.4 Looping trace at 5% capacity

At the smallest tested capacity, ARC has the strongest observed fixed-policy hit ratio on the looping trace, while adaptive performance is lower. This suggests that the window labels and configured map do not reliably select the best policy for this particular trace. It is possible that the active loop working set exceeds the cache capacity, but the report should not infer the exact cause without examining the generated trace and miss sequence.

B.5 Looping trace at 10% capacity

At 10% capacity, LFU and ARC exceed the adaptive controller in the recorded looping run. The controller switches policies frequently, which may disrupt useful policy state or reflect noisy feature profiles. The result demonstrates why a workload classifier must be evaluated on outcome metrics, not just on whether its labels appear plausible. A label that sounds correct can still lead to a poor policy choice.

B.6 Looping trace at 20% capacity

At the largest capacity, MRU performs best among the fixed policies in the observed run, while adaptive remains lower. This is a reminder that the best policy can depend on capacity as well as workload class. The current mapping does not use capacity as an input, so it cannot adapt the choice when the same access pattern behaves differently at different cache-to-working-set ratios.

B.7 Random trace at 5% capacity

The random trace produces low hit ratios across all policies at 5% capacity. The adaptive result is close to the strongest fixed baselines but does not establish a clear advantage. Under broad sampling, many blocks are unlikely to be revisited before eviction, so classification may have little leverage. A random workload with controlled skew would be more informative for evaluating frequency-based policy selection.

B.8 Random trace at 10% capacity

At 10% capacity, LFU and ARC are slightly ahead of adaptive in the observed random trace. The small difference should be interpreted descriptively because there is only one seed and no confidence interval. Repeating the experiment across seeds would show whether the ranking is stable or an artifact of a particular random sequence. The trace generator's sampling distribution should be documented in the result table.

B.9 Random trace at 20% capacity

A larger cache improves the hit ratio for all policies, but the adaptive result remains close to the best fixed baselines. This outcome is consistent with the idea that larger capacity captures more repeated identifiers regardless of policy. The relevant question for the adaptive layer is whether it adds benefit beyond this capacity effect, which requires paired differences against the strongest fixed policy at every capacity.

B.10 Mixed trace at 5% capacity

At 5% capacity, adaptive performs better than the fixed policies in the recorded mixed trace. The advantage may arise because the trace contains multiple access patterns and the controller can change policy at window boundaries. However, the result alone does not establish that clustering caused the improvement. A rule-only selector, a random policy selector, and an oracle mapping based on known generator phases would help isolate the value of clustering.

B.11 Mixed trace at 10% capacity

At 10% capacity, adaptive again exceeds the fixed baselines on the mixed trace. This is the clearest demonstration of the project's intended mechanism: one policy need not be best for all phases, so selecting among existing policies can be useful. The evidence is still limited to one deterministic trace. Phase durations, transition locations, and component proportions should be varied to determine whether the advantage is robust.

B.12 Mixed trace at 20% capacity

At 20% capacity, adaptive retains the best hit ratio among the tested policies in the observed mixed run. Larger capacity lowers the overall miss rate and reduces average modeled cost. A full evaluation should report whether the relative gain over the best fixed policy increases or decreases with capacity. If the gain vanishes at large capacity, the controller may be most useful under memory pressure.

B.13 Hit ratio and cost consistency

The reported average cost is computed directly from hit and miss counts. Under hit cost one and miss cost ten, the equation simplifies to 10 minus nine times hit ratio when hit ratio is expressed as a fraction. This identity provides a reproducibility check: a reported hit ratio should reproduce the reported cost within rounding error. It also makes clear that cost results are not independent evidence from hit ratio under this model.

B.14 Rounding policy

Tables round hit ratios to two decimal places and costs to three decimals for readability. Internal calculations should retain full precision until the final presentation. Small differences between policies may be sensitive to rounding, especially on the random workload. Therefore, the text should avoid declaring a decisive winner when values are nearly equal and should present raw counts or repeated-run uncertainty in a future version.

B.15 Capacity versus absolute cache slots

Percent capacity is normalized against unique identifiers in each trace, so the absolute number of slots can differ across workload families. A cross-workload chart can therefore compare relative capacity while hiding absolute cache size. The experiment should include both values in a detailed appendix or CSV. A second experiment with fixed absolute capacities would complement the current working-set-normalized design.

B.16 Mapping ablation

The policy map should be treated as a parameter under test. To evaluate it, swap one mapping at a time while keeping clustering labels fixed, then compare hit ratio and cost. A grid search across label-policy combinations can reveal whether the default mapping is close to the best mapping for each workload. Tuning must occur on training traces only, with final performance reported on held-out traces.

B.17 Oracle upper bound

An oracle upper bound could use known generator phases to select the best policy for each phase, based on separate calibration runs. It would not be deployable because it uses information unavailable to the live controller, but it would estimate how much headroom remains for better classification. If the oracle itself offers little improvement over a fixed policy, then more accurate clustering may not be worth the added complexity.

B.18 Rule-only baseline

A rule-only selector could use sequentiality, unique ratio, or concentration thresholds without K-Means. It would test whether unsupervised clustering contributes value beyond the engineered features. The baseline should use the same feature computation and policy map where possible, so the comparison isolates clustering overhead and decision quality. If it performs similarly with fewer switches, it may be the preferable design.

B.19 No-switch baseline

The strongest fixed policy for each workload can serve as a reference for the maximum benefit available from adaptation. In practice, that policy is not known in advance, but it is useful for post-hoc analysis. The adaptive controller should be compared against the best fixed baseline per trace, not only against an average of the four policies. This makes the performance gap more interpretable.

B.20 Random-switch baseline

A random-switch baseline could change among policies at the same approximate rate as the adaptive controller. If adaptive performs no better than random switching, the apparent benefit may come from incidental policy variation rather than meaningful labels. The random baseline should preserve the number of switches or the distribution of dwell times to make the comparison fair.

B.21 Stability score

Centroid distance can be used as a stability signal. A window close to its nearest centroid and far from the second-nearest centroid has a clearer assignment than a window lying near a boundary. The controller could refuse to switch when the distance gap is small. This does not create calibrated probability, but it provides a geometric ambiguity measure consistent with the K-Means model.

B.22 Hysteresis threshold

Hysteresis can require the new label to persist for two or more consecutive windows before a switch. This reduces rapid oscillation but adds delay. It should be tested against the current controller using the same traces, with hit ratio, cost, switch count, and adaptation time all reported. A good hysteresis threshold may differ between short bursts and long stable phases.

B.23 Minimum dwell time

A minimum dwell time forces a selected policy to remain active for a minimum number of requests or windows. It is simpler than learning a switch penalty and prevents excessive state reconstruction. However, it can lock the cache into a poor policy after an abrupt phase change. A sensitivity sweep should include no dwell time, one-window dwell time, and longer values, then compare both misses avoided and misses introduced by delayed switching.

B.24 Switching penalty

A cost-aware controller could assign a penalty to policy changes. A switch would occur only if the expected benefit over a future horizon exceeds the estimated state-transfer and runtime cost. The current project does not estimate this benefit, so the penalty cannot be calculated reliably yet. A future study could approximate expected cost from recent windows or maintain shadow-cache statistics for candidate policies.

B.25 Cache state invariants

Every policy switch should preserve capacity limits, avoid duplicate entries, and ensure that each resident block appears exactly once. Tests should check these invariants after every pairwise transition among LRU, LFU, MRU, and ARC. Membership preservation is not enough: the policy-specific metadata should also be initialized in a documented and deterministic manner. These tests are important because a bug in state migration can masquerade as a poor policy result.

B.26 Cache admission

Replacement policy and admission policy are distinct concepts. The current simulator focuses on replacement and admits missed blocks according to its policy implementation. Scan-resistant admission could change results substantially by preventing one-time blocks from entering the cache. A future extension should separate admission from eviction and test whether workload classification should choose a pair of admission and replacement strategies rather than a replacement policy alone.

B.27 Warm-up period

Hit ratio depends on the initial cache state. The supplied experiment begins with an empty cache unless the implementation initializes otherwise, so early requests incur compulsory misses. A warm-up protocol can either exclude an initial segment from metric calculation or explicitly report cold-start performance. The same warm-up rule must be applied to every policy. This distinction is especially relevant for short traces or small cache capacities.

B.28 Trace length

The default trace contains 50,000 requests, which provides enough windows for a first comparison but may not cover many independent phase transitions. Longer traces could reduce the influence of startup behavior and allow more stable estimates, but they also increase runtime. The study should compare multiple trace lengths and verify that relative policy rankings do not change merely because the evaluation includes a different number of phase repetitions.

B.29 Seed sensitivity

Seed 42 ensures a reproducible trace but samples only one realization of each workload family. For random and mixed traces, the exact request order can materially affect hit ratio. Multiple seeds should be selected before running experiments and retained even when results are unfavorable. The report should publish aggregate results and, ideally, per-seed CSV files so a reviewer can inspect variance and reproduce the summary.

B.30 Confidence intervals

Confidence intervals are not available from a single seed. With repeated traces, a paired interval for adaptive minus fixed-policy hit ratio can estimate uncertainty while controlling for trace difficulty. The same pairing should be used for cost. If the interval crosses zero, the report should state that the observed difference is not conclusive rather than treating the point estimate as proof of improvement.

B.31 Practical effect size

A percentage-point improvement in hit ratio should be accompanied by a relative change in miss count and modeled cost. For example, a small absolute gain can be meaningful when misses are expensive, but its value depends on actual latency and switch overhead. Effect sizes should be presented with uncertainty and operational context rather than focusing solely on whether a ranking changed.

B.32 Runtime decomposition

The total adaptive runtime should be decomposed into feature extraction, standardization, K-Means assignment, policy mapping, state transfer, and cache simulation. Clustering may be inexpensive relative to replaying 50,000 requests, while switching could dominate on a churn-heavy trace. Without decomposition, the single adaptation-time value cannot identify which component needs optimization.

B.33 Memory overhead

The current report emphasizes hit ratio and runtime but does not provide a measured memory-overhead comparison. A production controller would maintain feature windows, centroids, and policy metadata, while ARC also maintains ghost history. Memory use should be measured for each policy and for the adaptive controller, especially when the number of cache entries grows. A policy that saves misses but requires excessive metadata may not be beneficial under strict memory limits.

B.34 CPU overhead

Feature extraction scans each observation window and clustering compares vectors with centroids. The work is small for a handful of features but can still matter in high-throughput systems. The current Python simulation runtime is not a direct proxy for kernel-level CPU cost. Future work should report operations per request, CPU time per window, and the cost of state transfer under several cache sizes.

B.35 Multi-tenant effects

A shared cache can mix tenants with different locality patterns. One tenant's scan may evict another tenant's hot set, and a global classifier may label the aggregate stream as mixed even when each tenant has a stable pattern. A future extension should compare global and per-tenant feature windows, while accounting for isolation and memory partitioning. The current project does not implement tenant identity or fairness constraints.

B.36 Workload drift

A workload can change gradually rather than switch abruptly between discrete phases. K-Means labels impose hard cluster boundaries, so small feature changes near a centroid boundary can produce label flips. A drift-aware approach could track centroid distance over time or use a smoothing layer. The evaluation should include both abrupt and gradual transitions to determine whether the current window-based design is robust.

B.37 Missing telemetry

Real monitoring pipelines can contain missing or delayed observations. The current synthetic generator produces complete windows, so it does not test missing-feature behavior. A deployment-ready system should define whether missing values are imputed, whether a window is skipped, and whether the controller falls back to the current policy. It should also log the reason for every fallback to support diagnosis.

B.38 Safe fallback

If a cluster label cannot be mapped to a known policy, the controller should keep the current policy or use a configured safe default rather than failing or switching arbitrarily. Fallback behavior should be tested explicitly. A safe fallback also helps when feature extraction produces invalid values or the cluster model is unavailable. The current report identifies this as a future robustness requirement rather than claiming it is already implemented.

B.39 Reproducible plots

Figures should be generated directly from the exported metrics rather than manually edited values. Plot scripts should include axis labels, units, legend entries, and a note identifying the workload, capacity, and seed. The underlying CSV should be retained so a reviewer can recompute the charts. In this report, the plotted performance values are generated from the supplied simulator run.

B.40 Experimental logging

Each experiment should log configuration parameters, trace seed, unique-block count, actual cache capacity, policy name, hit and miss counts, average cost, switches, and runtime. Storing only a chart makes it difficult to verify values or analyze outliers. A machine-readable result table also supports multi-seed aggregation and later comparison of alternative feature sets or policy mappings.

B.41 Naming consistency

The terms ‘adaptive cache’, ‘adaptive controller’, and ‘adaptive policy selection’ should refer consistently to the same implementation. ARC is itself adaptive, so the name ADAPTIVE must not be confused with ARC. Tables and plots should use ‘ADAPTIVE’ for the project controller and ‘ARC’ for the baseline. This distinction matters when explaining results to reviewers and when discussing the difference between policy-internal adaptation and external workload classification.

B.42 Relevance of K-Means

K-Means is useful because it is lightweight and the resulting clusters can be inspected, but it is not inherently superior to threshold rules or other clustering methods. The project's use of K-Means should be justified through a baseline comparison, not by novelty alone. Future experiments could compare threshold classification, hierarchical clustering, and K-Means under the same features and report both classification stability and downstream cache performance.

B.43 Model selection objective

The best clustering objective is not necessarily the one that yields the best cache outcomes. K-Means minimizes within-cluster feature distance, while the system ultimately cares about hits, misses, and cost. A cluster partition can be compact yet map poorly to policies. Future work should evaluate cluster quality and end-to-end utility separately, and consider whether policy outcomes can be used to refine the mapping without overfitting to the test traces.

B.44 Policy regret

Regret can be defined as the difference between the adaptive controller's cost and the best fixed policy's cost on the same trace. This metric gives a direct measure of how much the controller loses or gains relative to a simple baseline. The best fixed policy is known only after the trace is complete, so it is an offline comparator rather than an online choice. Reporting regret across seeds would summarize both successful adaptation and failure cases.

B.45 Relevance of ARC comparison

Because ARC already adjusts its balance based on recent history, beating LRU alone would be insufficient evidence that external classification adds value. The adaptive controller should be compared with ARC and other strong baselines across all workload families. The observed looping and random results show why this comparison matters. A stronger study should explain when the external classifier provides information ARC does not already use internally.

B.46 Limits of the simulator

The simulator does not model write-back, dirty-page flushing, request concurrency, device queueing, CPU contention, power consumption, or actual storage latency. It also uses integer identifiers and synthetic distributions. These omissions are acceptable for a project prototype but bound its claims. The paper should describe the simulator as a controlled test harness and avoid language implying deployment on an operating-system cache.

B.47 Threat from manually chosen thresholds

The semantic labeling rules and policy mapping are manually selected. If the choices were made after inspecting the same results used for reporting, they may favor the tested traces. A more rigorous protocol would define the mapping before evaluation or tune it on separate development traces. All choices should be documented so a reviewer can distinguish design assumptions from empirical findings.

B.48 Threat from generated trace design

If the generator creates distinct patterns that are easy to separate, clustering may look effective even though real traces overlap. Conversely, a poor generator can make a sound classifier look weak. The generator should therefore be documented, validated against known properties, and supplemented with public traces. Sensitivity to working-set size, phase length, and skew should be reported.

B.49 Threat from evaluation metric

Hit ratio is useful but incomplete. It can reward a controller that achieves more hits while spending excessive CPU time switching policies. Average access cost partly captures miss penalties but omits switching overhead in its formula. Future work should define a combined objective that includes measured controller cost or report the two components separately so the trade-off is visible.

B.50 Interpretation of negative results

Underperforming a fixed policy is not a failed experiment; it identifies where the controller's assumptions do not hold. The looping result points toward label instability, mapping mismatch, or state migration as candidates for further study. The random result suggests limited benefit when locality is weak. Reporting these cases makes the evaluation more credible than reporting only the mixed-trace improvement.

B.51 Minimum viable next experiment

The next experiment should first remove the duplicated feature, then run multiple seeds for each workload and capacity. The policy mapping should be fixed before the test runs. For each run, export raw hit/miss counts and switch timelines. This small protocol would address two major issues—feature redundancy and seed sensitivity—without requiring a new model architecture or operating-system integration.

B.52 Public trace selection

Public traces should be selected to represent different reuse patterns and storage contexts. The report should state trace source, capture period, request fields, filtering, and whether block identifiers were remapped. Remapping must preserve locality relationships. Any missing fields or privacy transformations should be documented because they may affect sequentiality and reuse features.

B.53 Held-out evaluation

A proper held-out evaluation should keep complete traces out of mapping and feature-rule tuning. Splitting adjacent windows randomly can leak highly similar windows into training and testing because their histories overlap. Although the current project is not a supervised forecasting model, the same principle applies to selecting thresholds and policy mappings: the final performance estimate should come from traces not used to design the rules.

B.54 Fair compute budget

If one controller receives more preprocessing time or repeated tuning than another, the comparison can be biased. Fixed policies need little setup, while the adaptive controller incurs feature extraction and clustering. The paper should report total runtime and the number of tuning trials. A useful measure is benefit per unit of added runtime or memory, especially if the hit-ratio gain is small.

B.55 Policy transition testing

Every pairwise transition among LRU, LFU, MRU, and ARC should be tested under both empty and partially full cache states. Tests should verify resident count, uniqueness, capacity bound, and deterministic victim choice after migration. These tests separate algorithm correctness from performance. If state transfer is approximate by design, the paper should state that explicitly and evaluate whether the approximation materially changes hit ratio.

B.56 Result table design

A compact IEEE table should include policy names in rows and cache capacities in columns, or vice versa, while preserving enough precision to distinguish close results. The title belongs above the table and the explanatory note below it. Every figure should be referenced in the text before or near its appearance. Tables should use consistent percentage and cost units to avoid confusing ratios with absolute counts.

B.57 Reproduction commands

The source package should include one command that runs all workloads and capacities, writes machine-readable metrics, and creates the figures. The README should identify Python version and dependencies, and the experiment should record its random seed. If results are sensitive to environment or dependency versions, those details belong in the reproducibility note. This reduces the gap between the paper's tables and the code a reviewer can run.

B.58 Claim calibration

The strongest supported claim is that the implementation demonstrates an end-to-end adaptive selection pipeline and shows workload-dependent results on synthetic traces. It is not supported to claim that it always outperforms ARC, that it reduces real storage latency, or that K-Means is novel. Clear claim boundaries improve the paper's credibility and help define the next experiments.

B.59 From prototype to system

A production version would need a reliable telemetry source, bounded inference cost, policy transition safeguards, observability, rollback, and tests under concurrent workloads. It would also need to define whether the managed cache is a page cache, block cache, object cache, or application cache, because the replacement and admission mechanisms differ. These are substantial engineering steps beyond the current simulator.

B.60 Final interpretation

Taken together, the results show both potential and limits. The mixed trace demonstrates that external policy selection can help when the mapping aligns with phases. The looping and random traces demonstrate that classification does not guarantee better outcomes and that switching itself can be costly. The next stage should improve validation and stability before adding more complex models. This conclusion is consistent with the measured scope of the supplied code.

IX. LIMITATIONS AND THREATS TO VALIDITY (CONTINUED)

The principal limitation is external validity: synthetic traces and a fixed seed do not represent the full range of file-system or database behavior. The feature vector also contains a duplicated reuse-distance signal, semantic labels are heuristic, and policy selection is manually mapped. The observed improvement on the mixed trace is promising as a prototype result, but the weaker looping and random results show that the controller does not dominate established policies. A stronger evaluation must separate mapping design from test evaluation, repeat independent traces, and report uncertainty.

X. CONCLUSION AND FUTURE WORK

This paper presented an adaptive file-cache management prototype that classifies access windows and maps feature profiles to conventional replacement policies. It integrates synthetic trace generation, six feature measures, standardization, K-Means clustering, heuristic semantic labels, LRU/LFU/MRU/ARC baselines, window-boundary switching, and comparative metrics. In the default mixed-trace run, adaptive hit ratios were 16.97%, 30.20%, and 45.40% at 5%, 10%, and 20% capacity. The controller did not consistently outperform fixed policies on looping and random traces, and frequent switching coincided with increased runtime on looping access.

The defensible conclusion is conditional: workload-aware policy selection can help when the inferred phase and selected policy align, but classification does not guarantee improvement. Next steps are to remove redundant features, validate labels against known generator phases, add switch hysteresis, evaluate multiple seeds and held-out traces, compare against rule-only and oracle baselines, and measure real latency and controller overhead. Operating-system integration should follow only after the simulator results are stable and reproducible.

REFERENCES

[1] N. Megiddo and D. S. Modha, “ARC: A Self-Tuning, Low Overhead Replacement Cache,” in Proc. 2nd USENIX Conf. File and Storage Technologies (FAST), 2003, pp. 115–130.

[2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Pearson, 2015.

[3] N. Megiddo and D. S. Modha, “Outperforming LRU with an Adaptive Replacement Cache Algorithm,” Computer, vol. 37, no. 4, pp. 58–65, Apr. 2004.

[4] A. Vietri et al., “Driving Cache Replacement with ML,” in Proc. 10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage), 2018.

[5] J. MacQueen, “Some Methods for Classification and Analysis of Multivariate Observations,” in Proc. 5th Berkeley Symp. Mathematical Statistics and Probability, vol. 1, 1967, pp. 281–297.

[6] P. J. Denning, “The Working Set Model for Program Behavior,” Communications of the ACM, vol. 11, no. 5, pp. 323–333, 1968.

[7] A. S. Tanenbaum, “Cache memory and replacement strategies,” in Structured Computer Organization, 6th ed. Pearson, 2012.

[8] R. L. Mattson et al., “Evaluation Techniques for Storage Hierarchies,” IBM Systems Journal, vol. 9, no. 2, pp. 78–117, 1970.

[9] Supplied project materials, Adaptive File Cache Management Using Access Pattern Clustering: Python implementation, experiment scripts, README, and review presentations, 2026.

[10] Supplied simulator experiment, default configuration and random seed 42. Numerical values in this paper are outputs of the provided implementation; they are not external benchmark results.

Reproducibility: 50,000 requests; 5,000 possible block identifiers; seed 42; window size 1,000; four requested clusters; 100 K-Means iterations; eight restarts; capacities of 5%, 10%, and 20%; hit cost 1; miss cost 10. Results are simulator outputs, not external benchmark measurements.